

Jodie Butterworth

CPT_S 451

3-31-26

Week 10 Project Assignment: Database Implementation & System Validation

Objective

In this phase of your project, you will implement your database using SQL and demonstrate that your system works correctly under real-world conditions.

Your goal is to show that your database:

- Enforces all business rules
- Prevents invalid or inconsistent data
- Handles edge cases and failure scenarios
- Supports real-world operations through meaningful queries

Group Submission

- One submission per group
- All members are expected to contribute

PART 1: Database Implementation

Task

Implement your complete database using SQL.

You must include:

- CREATE TABLE statements
- Primary Keys (PK)
- Foreign Keys (FK)
- Appropriate constraints:
 - NOT NULL
 - UNIQUE
 - CHECK

- Composite keys (if applicable)
- ON DELETE / ON UPDATE rules

Table Name: _PROJECTCAMPAIGN _____

- Columns:
 - campaign_id — NUMBER — PK
 - title — VARCHAR2
 - start_date — DATE — NOT NULL
 - end_date — DATE — NOT NULL
 - funding_goal — DECIMAL — NOT NULL
 - description — VARCHAR2 — NOT NULL
- Primary Key: campaign_id
- Foreign Keys: none
- Description (what this table represents):
The campaigns of the funding site.

Table Name: _PROJECTUSER _____

- Columns:
 - user_id — NUMBER — PK
 - first_name — VARCHAR2 — NOT NULL
 - last_name — VARCHAR2 — NOT NULL
 - password_hash — VARCHAR2 — NOT NULL
 - role — VARCHAR2 — CHECK('user' or 'admin')
- Primary Key: user_id
- Foreign Keys: none
- Description (what this table represents):
The users of the funding site.

Table Name: _PROJECTTRANSACTION _____

- Columns:
 - transaction_id — NUMBER — PK
 - campaign_id — NUMBER — FK
 - user_id — NUMBER — FK
 - authorization_num — VARCHAR2 — NOT NULL
 - payment_type — VARCHAR2 — NOT NULL
 - payment_date — DATE — NOT NULL
 - amount — DECIMAL — NOT NULL, CHECK(amount>0)
- Primary Key: transaction_id
- Foreign Keys: campaign_id, user_id
- Description (what this table represents):
The transactions of the funding site.

Table Name: _ PROJECTEMAIL _____

- Columns:
 - user_id — NUMBER — FK, half of PK
 - email — VARCHAR2 — NOT NULL, half of PK
- Primary Key: user_id, email
- Foreign Keys: user_id
- Description (what this table represents):
The user emails.

Table Name: _ PROJECTSOCIALMEDIA _____

- Columns:
 - user_id — NUMBER — FK, half of PK
 - socialmedia — VARCHAR2 — NOT NULL, half of PK
- Primary Key: user_id, social media
- Foreign Keys: user_id
- Description (what this table represents):

The user social media accounts.

Table Name: _ PROJECTUSERCAMPAIGN _____

- Columns:
 - campaign_id — NUMBER — FK, half of PK
 - user_id — NUMBER — FK, half of PK
- Primary Key: campaign_id, user_id
- Foreign Keys: campaign_id, user_id
- Description (what this table represents):
Connects the PROJECTUSER and PROJECTCAMPAIGN tables.

PART 2: Business Rules Enforcement

Task

Identify **8–10 business rules** from your system and show how your database enforces them.

Example (for understanding)

In a parking system:

- A user should not have overlapping bookings
- A slot cannot be double-booked for the same time

Business Rule How It Is Enforced (Constraint / Design / Query)

Important

Focus on **real system constraints** — not generic statements.

1. For each transaction, the admins need to identify it if they need to contact the user or the payment company. Therefore, the field 'authorization_num' is set to NOT NULL to make sure the identifying number is recorded for each transaction.
2. For the transaction amount, it must be a number greater than 0. A negative number implies that the campaign should donate to the user, and a NULL amount means there isn't any money being paid. So, the 'amount' field is set to NOT NULL and has CHECK(amount>0).

3. If the campaign start date and end date are not set and displayed, users will not know when the campaign is running. The start and end date also set the time limits for collecting the transactions. Therefore, the 'start_date' and 'end_date' are set to NOT NULL so the users and the admins know when the campaign will be running.
4. In the campaign, the funding goal lets people know how much the campaign is trying to raise. Users would not know how much is needed in donations without this field, and admins would not know if the campaign is successful without a goal. So, the 'funding_goal' field is set to NOT NULL to ensure that everyone knows the campaign goal
5. In the campaign, the description tells the user what the campaign is for and why they should donate. The 'description' field is set to NOT NULL to make sure that every campaign is explained and easily identified.
6. In the user, the first name and last name are needed to identify and contact each user. The 'first_name' and 'last_name' are set to NOT NULL so that all the user names are recorded and connected to the user donations.
7. In the user, the password hash is used to log the user in. For access to the account to be secure, a password must be recorded and entered. For the password to be secure, it must be encrypted. The 'password_hash' field is set to NOT NULL to ensure that the account is secured.
8. In the user, the type of user is recorded to determine if they are a user or an admin. The user and admin get access to different features, and the view changes based on the person's role. The 'role' field has CHECK(role IN('user','admin')) to verify that user as one of these two types of users so that the program will allow access to the correct features.

PART 3: Edge Cases & Failure Handling

Task

Identify **6–8 realistic scenarios** where your system could fail or produce incorrect results.

Think About

- Duplicate or conflicting data
- Deleting records with dependencies
- Invalid references
- Overlapping or conflicting actions

- Simultaneous operations

Scenario What Can Go Wrong? How Your Database Handles It

1. Two users could have the same first and last names. This duplication is handled by giving each user a different user_id, so duplicate names will not cause problems.
2. A user could have multiple email addresses. This is handled by creating a separate table for the emails. Storing multiple email addresses in the same field is difficult to retrieve and edit. Storing the emails in a table where each one is connected to the user_id makes it easier to find and edit a user's emails.
3. A user could also have multiple social media accounts. Again, a separate table is made to handle the multiple entries per user. Storing the social media data in a separate table with a user_id FK makes it easier to find and edit a user's social media data.
4. Even though transactions would not exist without a user donating to the campaign, we want to keep financial records of the transactions after a user quits the site or the campaign is ended. Therefore, old user and campaign data is not deleted because the child table PROJECTTRANSACTION still needs it for financial records. RESTRICT is used when deleting is necessary for a row in campaign or user.
5. The PROJECTUSER and PROJECTCAMPAIGN tables are in a M : M relationship because one user can donate to many campaigns and one campaign can have donations from multiple users. A separate table with FK for both user_id and campaign_id is used to connect all the users to their campaigns and vice versa. The PK for the table is user_id and campaign_id together. This makes it easier correctly assign all the campaigns that a user has donated to and all the users that have given to a campaign.
6. The attribute for funding_received is calculated from all the transactions for that campaign. This could result in incorrect results if the wrong transactions are used. This is solved by querying the transactions with the campaign_id FK and using sum on them. This way only transactions from the right campaign will be added.
7. The attribute for title under campaign is useful for identifying a campaign, but depending on the logos or information in the description, it may be unnecessary or redundant. Therefore, it is possible to leave the title null in cases where it is not wanted.

PART 4: Data Operations (CRUD with Real Scenarios)

You must demonstrate how your system behaves during real operations.

Requirement

- At least **12 total queries**, covering:
 - INSERT
 - UPDATE
 - DELETE
 - SELECT

Important

Each query must represent a **real system action**, not a generic example.

Scenario 1:

User creates a new account.

Query:

```
INSERT INTO PROJECTUSER (  
    USER_ID,  
    FIRST_NAME,  
    LAST_NAME,  
    PASSWORD_HASH,  
    ROLE  
) VALUES ( 5,  
    'Bill',  
    'Purrin',  
    'password123',  
    'user' );
```

```
INSERT INTO PROJECTEMAIL (  
    USER_ID,  
    EMAIL  
) VALUES ( 5,  
    'billyp1@gmail.com' );
```

```
INSERT INTO PROJECTEMAIL (  
    USER_ID,  
    EMAIL  
) VALUES ( 5,  
    'billygoat1@gmail.com' );
```

```
INSERT INTO PROJECTSOCIALMEDIA (  
    USER_ID,  
    SOCIALMEDIA  
) VALUES ( 5,  
    'billybilly@facebook.com' );
```

Expected Behavior:

User data is added into the user database. The new email and social media data are added into the tables for email and social media. This is to create a new user in the database for the site.

Scenario 2:

Admin creates a new campaign.

Query:

```
INSERT INTO PROJECTCAMPAIGN (  
    CAMPAIGN_ID,  
    TITLE,  
    START_DATE,  
    END_DATE,  
    FUNDING_GOAL,  
    FUNDING_RECEIVED,  
    DESCRIPTION  
) VALUES ( 3,  
    'Better Living through Tacos',  
    TO_DATE('31-03-2026', 'DD-MM-YYYY'),
```

```
TO_DATE('16-06-2026', 'DD-MM-YYYY'),  
100.01,  
1.01,
```

'The sequel to my self-help book Insights on Extra Cheese. Better Living through Tacos explores the fast-food versus fine-dining taco in us all. I request donations from the small but dedicated food-based self-help book lovers.');

Expected Behavior:

Campaign data is added into the campaign database. This is to create a new campaign in the database for the site.

Scenario 3:

User makes a new donation for a campaign.

Query:

```
INSERT INTO PROJECTTRANSACTION (  
    TRANSACTION_ID,  
    CAMPAIGN_ID,  
    USER_ID,  
    AUTHORIZATION_NUM,  
    PAYMENT_TYPE,  
    PAYMENT_DATE,  
    AMOUNT  
) VALUES ( 4,  
    3,  
    5,  
    'TACO4',  
    'billy_credit_card',  
    TO_DATE('31-03-2026', 'DD-MM-YYYY'),  
    5.01 );
```

Expected Behavior:

Transaction data is added into the transaction database. This is to create a new transaction from the new user for the new campaign in the database for the site.

Scenario 4:

Admin removes a field from a table that is not needed.

Note: this is from the Oracle site: <https://www.oracletutorial.com/oracle-basics/oracle-drop-column/>

Query:

```
ALTER TABLE PROJECTCAMPAIGN
```

```
DROP COLUMN funding_received;
```

Expected Behavior:

The 'funding_received' field is dropped from the campaign table. This is a calculated value that can be done with queries, so it is not needed.

Scenario 5:

User changes his email address.

Note: emails are unique by the email providers already.

Query:

```
UPDATE PROJECTEMAIL
```

```
SET email = 'billynewemail@gmail.com'
```

```
WHERE email = 'billyp1@gmail.com';
```

Expected Behavior:

The email data is updated by specifying the email to change and the new email to replace it.

Scenario 6:

Admin changes the campaign description.

Query:

```
UPDATE PROJECTCAMPAIGN
```

```
SET description = 'The sequel to my self-help book Insights on Beans. Better Living through Tacos explores the fast-food versus fine-dining taco in us all. I request donations from the small but dedicated food-based self-help book lovers.'
```

WHERE campaign_id = 3;

Expected Behavior:

The campaign description is updated by specifying the campaign_id to identify the record and the field data to change with description.

Scenario 7:

Admin updates a campaign end date.

Query:

UPDATE PROJECTCAMPAIGN

SET end_date = TO_DATE('30-06-2026', 'DD-MM-YYYY')

WHERE campaign_id = 3;

Expected Behavior:

The campaign end-date is updated by specifying the campaign_id to identify the record and the field data to change with end_date.

Scenario 8:

Admin tries to delete a user.

Query:

DELETE FROM PROJECTUSER

WHERE user_id = 5;

Expected Behavior:

The attempt to delete the user fails because it is a parent. This is the result we want because we want to retain all records for financial purposes. (To delete records that are errors, delete the children first.)

Scenario 9:

Admin tries to delete a campaign.

Query:

DELETE FROM PROJECTCAMPAIGN

WHERE campaign_id = 3;

Expected Behavior:

The attempt to delete the campaign fails because it is a parent. This is the result we want because we want to retain all records for financial purposes. (To delete records that are errors, delete the children first.)

Scenario 10:

Admin queries the full name of the users.

Query:

```
SELECT first_name || ' ' || last_name AS full_name  
FROM PROJECTUSER;
```

Expected Behavior:

The full name of the users are given from the first_name and last_name data being joined together.

Scenario 11:

Admin queries the total donations for a campaign.

Query:

```
SELECT sum(amount) total_donations  
FROM PROJECTTRANSACTION  
WHERE campaign_id = 3;
```

Expected Behavior:

The total donations for a campaign are added up using the campaign_id to get the amounts from the transaction table.

Scenario 12:

User asks to list their donation amounts and dates.

Query:

```
SELECT amount, payment_date  
FROM PROJECTTRANSACTION  
WHERE user_id = 5;
```

Expected Behavior:

The donation amounts and dates for that user are displayed up using the user_id to get the amounts and dates from the transaction table.

Examples

- Create a booking
- Update booking time
- Cancel a reservation
- Delete a user (what happens to related data?)

PART 5: Advanced Retrieval Queries

Design **10 real-world queries** that:

- Use **JOINS (mandatory)**
- Involve multiple tables
- Reflect actual system usage

Example Scenarios

- Find all available resources for a given time range
- Identify active users or transactions
- Detect conflicts or overlaps
- Analyze usage patterns

Query #: 1

Scenario:

List donations for each user with the user names and amounts

SQL Query:

```
SELECT first_name, last_name, amount FROM PROJECTUSER, PROJECTTRANSACTION  
WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id;
```

Why JOIN is required:

Inner join to list the names from the user table with the amounts from the transaction table be the user_id PK and user_id FK.

Expected Output:

The user's first and last name with the donation amounts.

Query #: _2__

Scenario:

List donations for each campaign with the title and amounts

SQL Query:

```
SELECT title, amount FROM PROJECTCAMPAIGN, PROJECTTRANSACTION  
WHERE PROJECTCAMPAIGN.campaign_id = PROJECTTRANSACTION.campaign_id;
```

Why JOIN is required:

Inner join to list the titles from the campaign table with the amounts from the transaction table be the campaign_id PK and campaign_id FK.

Expected Output:

The campaign title with the donation amounts.

Query #: _3__

Scenario:

Display amount and payment date of donation with the user last name

SQL Query:

```
SELECT amount, payment_date, last_name FROM PROJECTTRANSACTION,  
PROJECTUSER  
WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id;
```

Why JOIN is required:

Inner join to the last name from the user table with the amounts and payment dates from the transaction table using user_id PK and user_id FK.

Expected Output:

The donation amounts, dates with the user who donated last name.

Query #: _4__

Scenario:

Display user first and last name with their emails

SQL Query:

```
SELECT first_name, last_name, email FROM PROJECTUSER, PROJECTEMAIL  
WHERE PROJECTUSER.user_id = PROJECTEMAIL.user_id;
```

Why JOIN is required:

Inner join to the first name and last name from the user table with the emails from the email table using user_id PK and user_id FK.

Expected Output:

The name of the user with their emails.

Query #: _5__

Scenario:

Display user first and last name with their social media

SQL Query:

```
SELECT first_name, last_name, socialmedia FROM PROJECTUSER,  
PROJECTSOCIALMEDIA
```

```
WHERE PROJECTUSER.user_id = PROJECTSOCIALMEDIA.user_id;
```

Why JOIN is required:

Inner join to the first name and last name from the user table with the socialmedia from the social media table using user_id PK and user_id FK.

Expected Output:

The name of the user with their social media accounts.

Query #: _6__

Scenario:

Display user name if the user gave a donation greater than 20 dollars.

SQL Query:

```
SELECT first_name, last_name, amount FROM PROJECTUSER, PROJECTTRANSACTION
```

```
WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id
```

```
AND PROJECTTRANSACTION.amount > 20;
```

Why JOIN is required:

Inner join to the first name and last name from the user table with the amount from the transaction table using user_id PK and user_id FK.

Expected Output:

The name of the users who have given a donation greater than 20 dollars.

This thing is a beast, and I still haven't done the classwork. I'm just done here.

Important

These should reflect real application behavior.

PART 6: Validation with Sample Data

Task

Provide sample data and demonstrate that your queries work correctly.

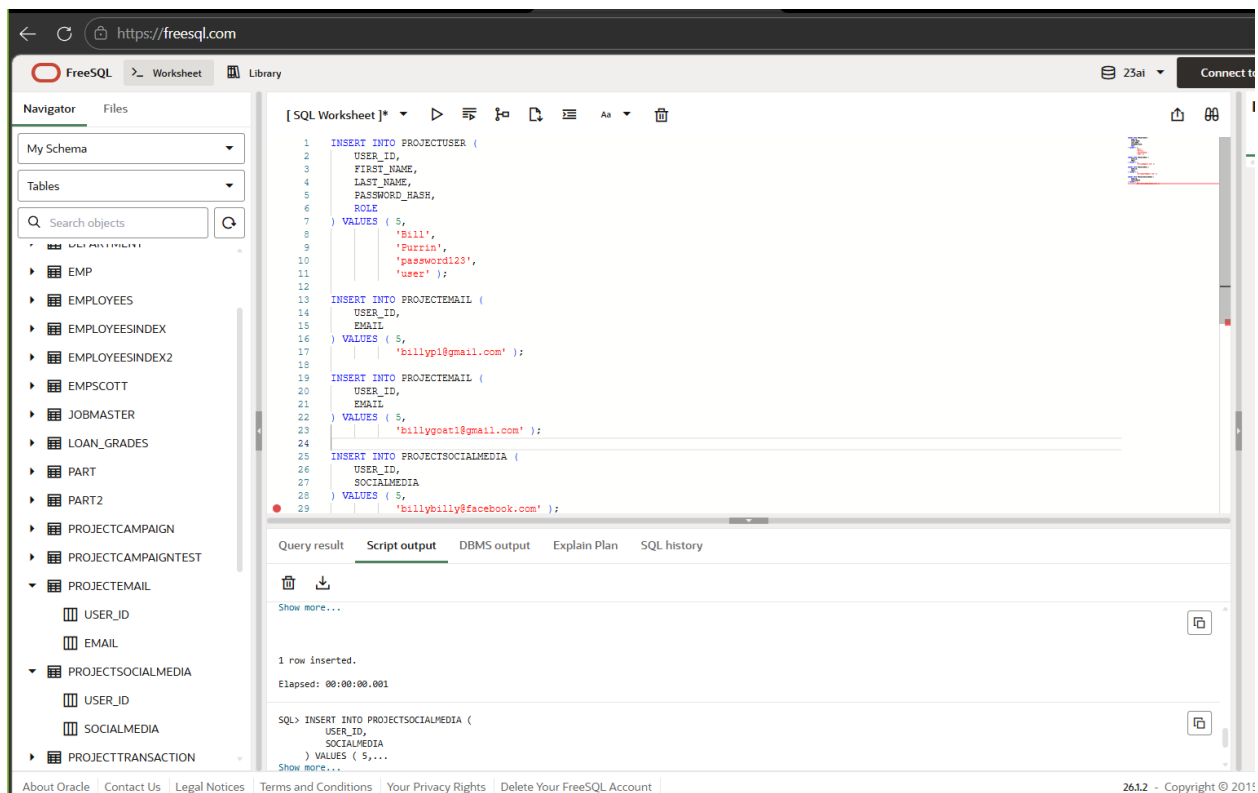
You must:

- Insert meaningful test data
- Show outputs for key queries (screenshots or result tables)

Expectation

- Data should reflect realistic scenarios
- Queries should produce correct and meaningful results

I've been doing this. I'll show the screenshots.



New user added.

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' sidebar shows a tree view of a database schema named 'My Schema'. The 'Tables' section is expanded, listing various tables including EMP, EMPLOYEES, EMPLOYEESINDEX, EMPLOYEESINDEX2, EMPSCOTT, JOBMASTER, LOAN_GRADES, PART, PART2, PROJECTCAMPAIGN, PROJECTCAMPAIGNTEST, PROJECTEMAIL, PROJECTSOCIALMEDIA, and PROJECTTRANSACTION. The main workspace, titled '[SQL Worksheet]', contains an SQL insert statement for the 'PROJECTCAMPAIGN' table. The statement includes fields for CAMPAIGN_ID, TITLE, START_DATE, END_DATE, FUNDING_GOAL, FUNDING_RECEIVED, and DESCRIPTION. The values provided are: CAMPAIGN_ID = 3, TITLE = 'Better Living through Tacos', START_DATE = '2026-03-31', END_DATE = '2026-06-16', FUNDING_GOAL = 100.01, and FUNDING_RECEIVED = 1.01. The description is a multi-line text about a book. Below the SQL editor, the 'Script output' tab is active, showing the executed SQL statement and the result: '1 row inserted.' The elapsed time for the operation is '00:00:00.009'.

```
1 INSERT INTO PROJECTCAMPAIGN (
2   CAMPAIGN_ID,
3   TITLE,
4   START_DATE,
5   END_DATE,
6   FUNDING_GOAL,
7   FUNDING_RECEIVED,
8   DESCRIPTION
9 ) VALUES ( 3,
10            'Better Living through Tacos',
11            TO_DATE('31-03-2026', 'DD-MM-YYYY'),
12            TO_DATE('16-06-2026', 'DD-MM-YYYY'),
13            100.01,
14            1.01,
15            'The sequel to my self-help book Insights on Extra Cheese. Better Living through Tacos explores the fast-food versus fine-d
16
17
```

SQL> INSERT INTO PROJECTCAMPAIGN (
CAMPAIGN_ID,
TITLE,
START_DATE,...
Show more...

1 row inserted.

Elapsed: 00:00:00.009

New campaign added.

FreeSQL | Worksheet | Library | 23al | Connect

Navigator | Files

My Schema

Tables

Search objects

EMP

EMPLOYEES

EMPLOYEESINDEX

EMPLOYEESINDEX2

EMPSCOTT

JOBMASTER

LOAN_GRADES

PART

PART2

PROJECTCAMPAIGN

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

USER_ID

SOCIALMEDIA

PROJECTTRANSACTION

```
1 INSERT INTO PROJECTTRANSACTION (
2   TRANSACTION_ID,
3   CAMPAIGN_ID,
4   USER_ID,
5   AUTHORIZATION_NUM,
6   PAYMENT_TYPE,
7   PAYMENT_DATE,
8   AMOUNT
9 ) VALUES (4,
10  .....3,
11  .....5,
12  ..... 'TAC04',
13  ..... 'billy_credit_card',
14  ..... TO_DATE('31-03-2026', 'DD-MM-YYYY'),
15  ..... 5.01);
16
17
```

Query result | Script output | DBMS output | Explain Plan | SQL history

SQL> INSERT INTO PROJECTTRANSACTION (
TRANSACTION_ID,
CAMPAIGN_ID,
USER_ID,...
Show more...

1 row inserted.

Elapsed: 00:00:00.006

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account

2612 - Copyright © 201

New transaction added.

FreeSQL Worksheet

My Schema

Tables

Search objects

EMPLOYEESINDEX2

EMPSCOTT

JOBMASTER

LOAN_GRADES

PART

PART2

PROJECTCAMPAIGN

CAMPAIGN_ID

TITLE

START_DATE

END_DATE

FUNDING_GOAL

DESCRIPTION

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

```
1 DELETE FROM PROJECTCAMPAIGN
2 WHERE campaign_id = 3;
```

Query result Script output DBMS output Explain Plan SQL history

ORA-02292: integrity constraint (300IE_BUTTERWORTH_SCHEMA_4GK35.FK_USERSOCIALMEDIA) violated - child record found

<https://docs.oracle.com/error-help/db/ora-02292/>

Error at Line: 4 Column: 0

SQL> DELETE FROM PROJECTCAMPAIGN

WHERE campaign_id = 3

ORA-02292: integrity constraint (300IE_BUTTERWORTH_SCHEMA_4GK35.FK_CAMPAIGN) violated - child record found

<https://docs.oracle.com/error-help/db/ora-02292/>

Error at Line: 4 Column: 0

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account

26.1.2 - Copyright © 20

Delete of campaign fails, as planned.

FreeSQL | Worksheet | Library | 23:01 | Connected

Navigator | Files

My Schema

Tables

Search objects

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

PROJECTTRANSACTION

PROJECTTRANSACTIONTEST

PROJECTUSER

USER_ID

FIRST_NAME

LAST_NAME

PASSWORD_HASH

ROLE

PROJECTUSERCAMPAIGN

SCOTT

SCOTT2

SHIPMENT

STUDENT

```
[SQL Worksheet]*  
1 SELECT first_name || ' ' || last_name AS full_name  
2 FROM PROJECTUSER;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.004 seconds

	FULL_NAME
1	Test User
2	Test2 User2
3	Bill Purrin

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 26.1.2 - Copyright © 2020

Query the full name of the users.

FreeSQL | Worksheet | Library | 23ai | Connect

Navigator | Files

My Schema

Tables

Search objects

- PROJECTCAMPAIGNTEST
- PROJECTEMAIL
 - USER_ID
 - EMAIL
- PROJECTSOCIALMEDIA
- PROJECTTRANSACTION
- PROJECTTRANSACTIONTEST
- PROJECTUSER
 - USER_ID
 - FIRST_NAME
 - LAST_NAME
 - PASSWORD_HASH
 - ROLE
- PROJECTUSERCAMPAIGN
- SCOTT
- SCOTT2
- SHIPMENT
- STUDENT

```
1 SELECT sum(amount) total_donations
2 FROM PROJECTTRANSACTION
3 WHERE campaign_id = 3;
```

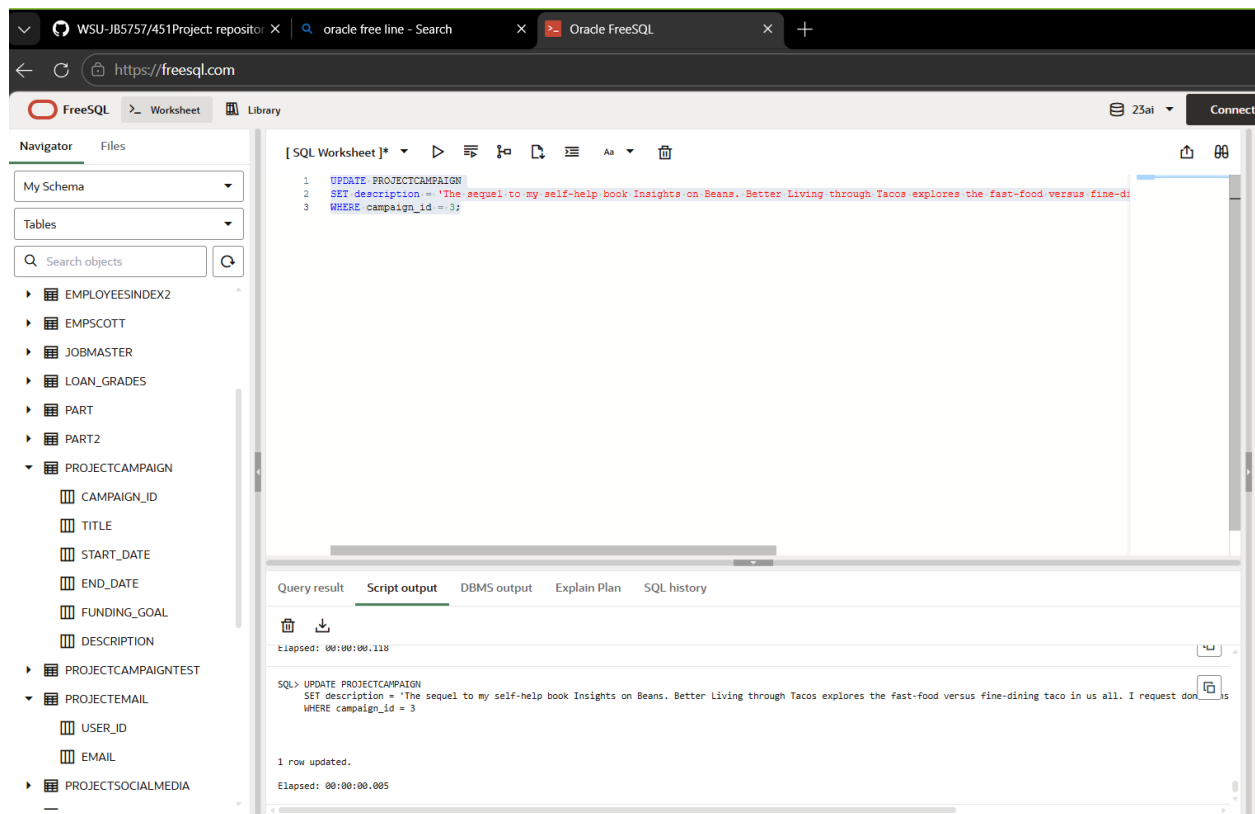
Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.005 seconds

TOTAL_DONATIONS	
1	5.01

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 2612 - Copyright © 201

Query the sum of the donations for a campaign.



Update the campaign description.

FreeSQL | Worksheet | Library | 23ai | Connect

Navigator | Files

My Schema

Tables

Search objects

EMPLOYEEINDEX2

EMPSCOTT

JOBMASTER

LOAN_GRADES

PART

PART2

PROJECTCAMPAIGN

CAMPAIGN_ID

TITLE

START_DATE

END_DATE

FUNDING_GOAL

DESCRIPTION

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

[SQL Worksheet]*

```
1 UPDATE PROJECTCAMPAIGN
2 SET end_date = TO_DATE('30-06-2026', 'DD-MM-YYYY')
3 WHERE campaign_id = 3;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

https://docs.oracle.com/error-help/db/ora-01839/

Error at Line: 4 Column: 0

SQL> UPDATE PROJECTCAMPAIGN
SET end_date = TO_DATE('30-06-2026', 'DD-MM-YYYY')
WHERE campaign_id = 3

1 row updated.

Elapsed: 00:00:00.005

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account

26.1.2 - Copyright © 20

Update the campaign end date.

WSU:BS757/451/Project: repository x oracle free line Search Oracle FreeSQL

FreeSQL | Worksheet | Library | 23:51 | Connect to the Database | Help and Feedback | Sign Out

Navigator | Files

My Schema

Tables

Search objects

PAYMENT_DATE

AMOUNT

PROJECTTRANSACTIONTEST

PROJECTUSER

USER_ID

FIRST_NAME

LAST_NAME

PASSWORD_HASH

ROLE

PROJECTUSERCAMPAGN

CAMPAGN_ID

USER_ID

SCOTT

SCOTT2

SHIPMENT

STUDENT

STUDENT_DOB

SUPPLIER

```
1 SELECT first_name, last_name, amount FROM PROJECTUSER, PROJECTTRANSACTION
2 WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.007 seconds

	FIRST_NAME	LAST_NAME	AMOUNT
1	Test2	User2	50.02
2	Bill	Purkin	5.01
3	Test	User	10.01

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account

20.1.2 - Copyright © 2015, 2020 Oracle and/or its affiliates All rights reserved

10:44 PM 3/31/2026

Library

Community | My Content | My Favorites | Recently

Search

Tutorial

Creating Tables: Databases for...

An introduction to creating tables and the types of these available in Oracle Database.

Created 8 years ago

1 likes 10.3K executions

Tutorial

Joining Tables: Databases for...

An introduction to the join types available in Oracle Database.

Created 7 years ago

2 likes 2.2K executions

Tutorial

Querying and Filtering Rows:...

User names with their donation amounts.

FreeSQL | Worksheet | Library | 23al | Connect

Navigator | Files

My Schema

Tables

Search objects

JOBCMASTER

LOAN_GRADES

PART

PART2

PROJECTCAMPAIGN

CAMPAIGN_ID

TITLE

START_DATE

END_DATE

FUNDING_GOAL

DESCRIPTION

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

PROJECTTRANSACTION

TRANSACTION_ID

SQL Worksheet]*

```
1 SELECT title, amount FROM PROJECTCAMPAIGN, PROJECTTRANSACTION
2 WHERE PROJECTCAMPAIGN.campaign_id = PROJECTTRANSACTION.campaign_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.11 seconds

	TITLE	AMOUNT
1	campaigntest2	50.02
2	Better Living through	5.01
3	campaigntest1	10.01

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 2612 - Copyright © 201

Campaign titles with the amounts for them.

FreeSQL | Worksheet | Library | 25ai | Connect to

Navigator | Files

My Schema

Tables

Search objects

PROJECTCAMPAIGNTEST

PROJECTEMAIL

USER_ID

EMAIL

PROJECTSOCIALMEDIA

PROJECTTRANSACTION

TRANSACTION_ID

CAMPAIGN_ID

USER_ID

AUTHORIZATION_NUM

PAYMENT_TYPE

PAYMENT_DATE

AMOUNT

PROJECTTRANSACTIONTEST

PROJECTUSER

USER_ID

FIRST_NAME

LAST_NAME

[SQL Worksheet]*

```
1 SELECT amount, payment_date, last_name FROM PROJECTTRANSACTION, PROJECTUSER
2 WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.006 seconds

	AMOUNT	PAYMENT_DATE	LAST_NAME
1	10.01	3/15/2026, 12:00:01	User
2	50.02	3/15/2036, 12:00:01	User2
3	5.01	3/31/2026, 12:00:01	Purrin

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 26.1.2 - Copyright © 2015

User last name with their donation amount and date.

FreeSQL | Worksheet | Library | 25ai | Connect

Navigator | Files

My Schema

Tables

Search objects

PAYMENT_DATE

AMOUNT

PROJECTTRANSACTIONTEST

PROJECTUSER

USER_ID

FIRST_NAME

LAST_NAME

PASSWORD_HASH

ROLE

PROJECTUSERCAMPAIGN

CAMPAIGN_ID

USER_ID

SCOTT

SCOTT2

SHIPMENT

STUDENT

STUDENT_DOB

SUPPLIER

[SQL Worksheet]*

```
1 SELECT first_name, last_name, amount FROM PROJECTUSER, PROJECTTRANSACTION
2 WHERE PROJECTUSER.user_id = PROJECTTRANSACTION.user_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.007 seconds

	FIRST_NAME	LAST_NAME	AMOUNT
1	Test2	User2	50.02
2	Bill	Purrin	5.01
3	Test	User	10.01

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account

26.1.2 - Copyright © 20

User name with their emails.

FreeSQL Worksheet

My Schema

Tables

Search objects

PROJECTCAMPAIGN

- CAMPAIGN_ID
- TITLE
- START_DATE
- END_DATE
- FUNDING_GOAL
- DESCRIPTION

PROJECTCAMPAIGNTEST

PROJECTEMAIL

- USER_ID
- EMAIL

PROJECTSOCIALMEDIA

- USER_ID
- SOCIALMEDIA

PROJECTTRANSACTION

- TRANSACTION_ID
- CAMPAIGN_ID
- USER_ID

```
1 SELECT first_name, last_name, socialmedia FROM PROJECTUSER, PROJECTSOCIALMEDIA
2 WHERE PROJECTUSER.user_id = PROJECTSOCIALMEDIA.user_id
```

Query result

Script output DBMS output Explain Plan SQL history

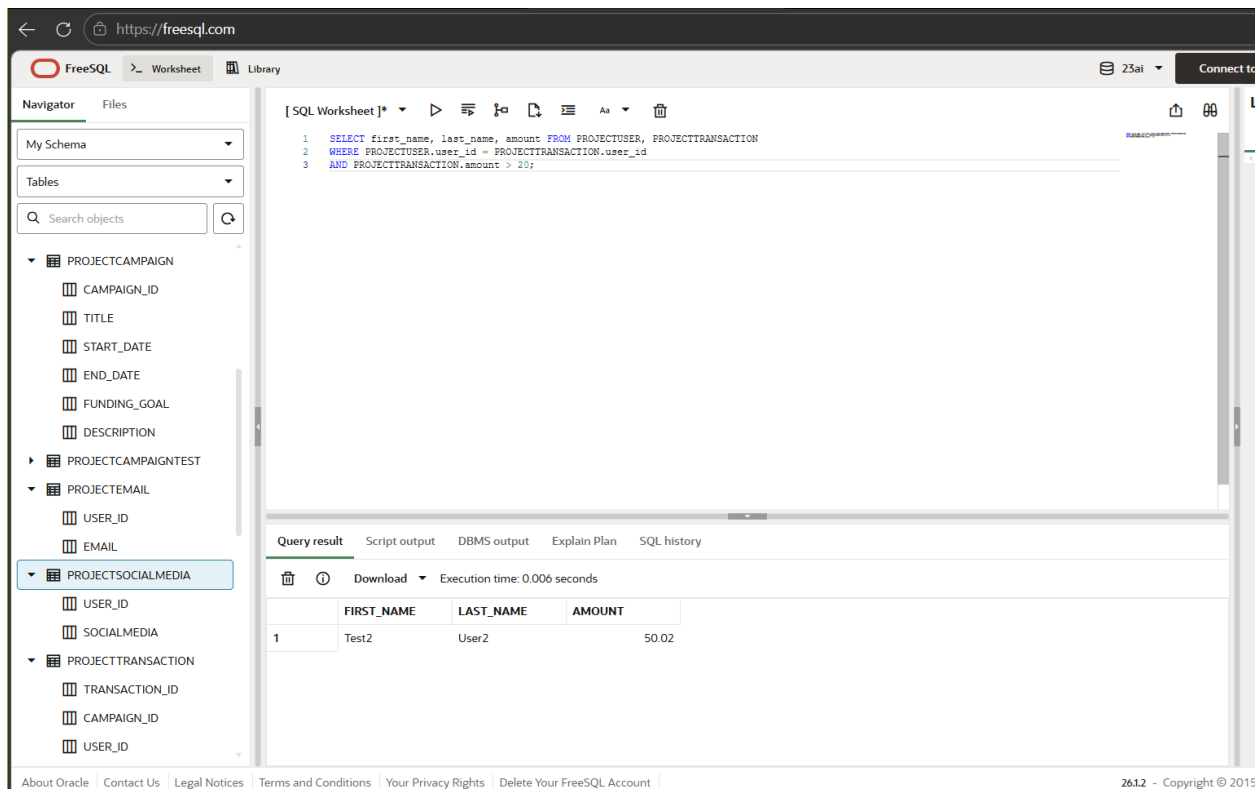
Download Execution time: 0.009 seconds

	FIRST_NAME	LAST_NAME	SOCIALMEDIA
1	Bill	Purkin	billybilly@facebook.c

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account

26.12 - Copyright © 201

User names with their social media accounts.



User names that gave a donation over 20 and the amount.

PART 7: System Integrity Reflection

Answer briefly:

1. What is the most critical business rule in your system?
Maintain accurate data that is complete and accessible.
2. What was the hardest edge case to handle?
The donation list for users requires all the tables.
3. What happens if two users perform the same action at the same time?
The users all have user_id's unique to them. Both perform the action, but there is no overlap.
4. How does your database prevent invalid states?
It restricts deletions for anything with children. All users, transactions and campaigns have their own unique ids. Most fields are NOT NULL constraint.

Submission Guidelines

- Submit as a **single document (PDF)**

- Clearly organized sections
- Proper formatting (tables, headings, SQL blocks)
- Include query outputs where required

Evaluation Criteria

Criteria	Marks
Database design & constraints	5
Business rules enforcement	5
Edge case handling	5
CRUD operations (real scenarios)	5
Advanced JOIN queries	5
Clarity & completeness	5

Final Note

This assignment is about **thinking like a system designer**.

Ask yourself:

- Can my system break?
Can invalid data enter?
- Do my queries reflect real usage?